How I Added Performance Monitoring to My MCP Server with AppSignal

This article is available as a blog post online at https://www.vmugdha.in/blog/appsignal-mcp-monitoring.html.

Scaling an enterprise-grade app requires more than shipping features. It depends on clear visibility into performance. For my Local Media Search MCP server, that meant adding real application performance monitoring (APM) instead of relying only on container logs from the Hugging Face Space where the server is deployed.

The MCP server is a Python and Gradio application that uses SigLIP to encode text queries, search a FAISS index, and generate embeddings for images and video frames during indexing. These CPU-intensive steps push the limits of a free-tier Hugging Face Space. While the container logs provide detailed event-level output, monitoring the MCP server requires additional structured metrics and historical performance data.

To get that level of observability, I integrated AppSignal with the MCP server. This post explains why I chose AppSignal, how I instrumented the server to send the performance data to AppSignal, and the performance insights now available through AppSignal dashboards.

Why AppSignal?

When evaluating APM tools, I had specific requirements for this project:

Python + OpenTelemetry support: AppSignal's Python SDK is built on OpenTelemetry, allowing standard OpenTelemetry instrumentation libraries to work alongside AppSignal's helper functions. These libraries and helper functions could be easily added to the MCP server's async Python logic.

Custom metrics without infrastructure overhead: AppSignal's `set_gauge()` and `increment_counter()` helpers let me track custom metrics with just a function call. No need to run a separate metrics server or additional services to deploy.

Error tracking with context: When SigLIP inference fails or a file becomes inaccessible, I needed the stack trace *plus* the context of what operation was running.

AppSignal's `send_error()` captures both.

Anomaly detection and alerting: I wanted to know when search latency spikes above normal, not just see it in a graph later. AppSignal's trigger system lets me set thresholds and get notified.

Uptime monitoring included: Since the Hugging Face Space can go to sleep, I needed uptime checks. AppSignal includes this in the same dashboard, so I don't need yet another monitoring service.

The Instrumentation

The MCP server requires instrumentation, that is, adding code that gathers app performance data to send to AppSignal. This touched three files: `requirements.txt`, `app.py`, and `ai_indexer.py`.

Dependencies

```
# requirements.txt
appsignal
opentelemetry-instrumentation-starlette
```

The `appsignal` package includes all the Python helpers. The Starlette instrumentation auto-captures HTTP requests to Gradio's underlying FastAPI app.

Configuration

I chose inline configuration over the `__appsignal__.py` file approach (see Bug Fixes). On Hugging Face Spaces, auto-discovery of config files was unreliable, and inline configuration made debugging easier:

```
# app.py
from appsignal import Appsignal, set_category, send_error, set_gauge, incre
from opentelemetry.instrumentation.starlette import StarletteInstrumentor
from opentelemetry import trace
from dotenv import load_dotenv

# Load .env BEFORE AppSignal initialization
load_dotenv()

appsignal_client = Appsignal(
    active=True,
    name="MediaSearchMCP",
    push_api_key=os.getenv("APPSIGNAL_PUSH_API_KEY", ""),
    environment=os.getenv("APPSIGNAL_APP_ENV", "development"),
    enable_host_metrics=True,
)
appsignal_client.start()
```

The `enable_host_metrics=True` flag tells AppSignal to collect CPU, memory, and disk usage from the container. This is useful for spotting when the free-tier Space is running out of resources. Note that `load_dotenv()` must run before `Appsignal()` is initialized (see Bug Fixes).

I also instrumented Gradio's internal FastAPI app to capture HTTP requests:

```
# Apply Starlette instrumentation to Gradio's internal FastAPI app
StarletteInstrumentor().instrument_app(demo.app)
```

For Hugging Face Spaces, I set two secrets in the Space settings:

- `APPSIGNAL_PUSH_API_KEY`: the API key from AppSignal
- `APPSIGNAL_APP_ENV`: set to `production`

Custom OpenTelemetry Spans for MCP Tools

I instrumented every MCP tool to have its own OpenTelemetry span, a self-contained record of every operation of that tool, including relevant attributes of timing, metadata, and relationship with other operations in the same trace. Here's the pattern I used for `semantic_search`:

```
tracer = trace.get_tracer(__name__)

async def semantic_search(query: str, media_type: str = "all", top_k: int =
    start_time = time.time()
    increment_counter("semantic_search.calls", 1)

    with tracer.start_as_current_span("semantic_search") as span:
        set_category("mcp_tool.semantic_search")
        span.set_attribute("search.query", query)
        span.set_attribute("search.media_type", media_type)
        span.set_attribute("search.top_k", top_k)

        try:
            results = indexer.search(query, top_k=top_k)

            # Track result metrics
            span.set_attribute("search.result_count", len(results))
            set_gauge("semantic_search.result_count", len(results))

            if results:
                similarities = [r["similarity"] for r in results]
                set_gauge("semantic_search.top_similarity", max(similaritie
                set_gauge("semantic_search.avg_similarity", sum(similaritie
            else:
                increment_counter("semantic_search.zero_results", 1)

            duration_ms = (time.time() - start_time) * 1000
            set_gauge("semantic_search.duration_ms", duration_ms)

            return results

        except Exception as e:
            increment_counter("semantic_search.errors", 1)
            send_error(e)
            raise
```

The `set_category()` call is AppSignal-specific. It groups spans in the dashboard so I can see all `mcp_tool.*` operations together.

I applied the same pattern to all four MCP tools:

Span Name	Key Attributes
`semantic_search`	`search.query`, `search.media_type`, `search.top_k`, `search.result_count`

`get_media_details`	`media.file_path`, `media.type`
`reindex_media`	`reindex.force`, `reindex.file_count`
`get_index_stats`	`stats.total_files`, `stats.images`, `stats.videos`

Custom OpenTelemetry Spans for the AI Indexer

I also instrumented the core AI operations in `ai_indexer.py` with their own OpenTelemetry spans. Here's the instrumentation for the `search` operation:

```python
# ai_indexer.py
from opentelemetry import trace
from appsignal import send_error

tracer = trace.get_tracer(__name__)

def search(self, query: str, top_k: int = 5):
    with tracer.start_as_current_span("indexer_search") as span:
        span.set_attribute("search.query", query)
        span.set_attribute("search.top_k", top_k)

        try:
            # Get text embedding using SigLIP
            with tracer.start_as_current_span("get_image_embedding"):
                # High-frequency operation, kept lightweight (no attributes
                query_embedding = self._encode_text(query)

            # FAISS search
            results = self._search_index(query_embedding, top_k)
            span.set_attribute("search.result_count", len(results))
            return results

        except Exception as e:
            send_error(e)
            raise
```

This creates nested traces. When a user performs a search, I can see the full call chain:

```
POST /api/predict (auto-instrumented by Starlette)
  └── semantic_search (app.py)
        └── indexer_search (ai_indexer.py)
              └── get_image_embedding (ai_indexer.py)
```

Error Reporting

Every `except` block now reports to AppSignal:

```
try:
    self.model = SigLIPModel.from_pretrained(model_name)
except Exception as e:
    send_error(e)  # Reports to AppSignal with full stack trace
    raise
```

I added `send_error()` calls to:

- Model loading failures
- Video frame extraction errors
- FAISS index save/load failures
- File access errors in media processing
- All MCP tool exception handlers

Custom Metrics Summary

Here's the full list of custom metrics the server now reports:

Gauges (point-in-time values):

- `semantic_search.duration_ms` : end-to-end search latency
- `semantic_search.result_count` : results returned per search
- `semantic_search.top_similarity` : highest similarity score
- `semantic_search.avg_similarity` : average similarity across results
- `reindex.duration_ms` : full reindex time
- `reindex.file_count` : files after reindex
- `index.total_files` , `index.images` , `index.videos` : current index stats

Counters (cumulative events):

- `semantic_search.calls` : total searches
- `semantic_search.zero_results` : searches with no matches
- `semantic_search.errors` : failed searches
- `reindex.calls` , `reindex.errors` : reindex operations
- `get_index_stats.calls` : stats queries

Configuring AppSignal Dashboard

With data flowing in, I configured the AppSignal side to make it actionable.

Custom Metrics Dashboard

I added a custom dashboard in AppSignal, and configured in it custom metrics reported by the MCP server.

AppSignal custom dashboard titled 'Search Performance Metrics' displaying four line-graph panels arranged in a 2x2 grid: Semantic Search Duration showing latency spikes up to 60ms over time, Semantic Search Similarity tracking result quality scores, Semantic Search Results displaying the number of results returned per query, and Reindex Duration showing time spent on reindex operations. The graphs reveal latency spikes around 17:00 correlating with cold starts after the Hugging Face Space wakes from sleep.

For example, to visualize the `semantic_search.duration_ms` gauge over time, I configured the following custom metric:

- **Chart title:** "Semantic Search Duration"
- **Metrics:** Select the `semantic_search.duration_ms` gauge
- **Graph display:** Line graph works best for time-series latency data
- **Data format:** Duration (displays values in milliseconds/seconds)
- **NULL values:** "Draw NULL as 0"

AppSignal Graph Builder configuration interface showing the setup panel on the left with options for chart title 'Semantic Search Duration', metrics dropdown selecting 'semantic_search.duration_ms' gauge, graph display type set to line graph for time-series data, data format as Duration in milliseconds/seconds, and NULL value handling option set to 'Draw NULL as 0'. The right side displays a real-time preview of the configured line graph with the actual metric data.

The chart preview on the right shows real-time data as per the configured graph.

The "Search Performance Metrics" dashboard displays four key gauges tracking search performance over time: **Semantic Search Duration** (latency spikes up to 60ms), **Semantic Search Similarity** (result quality scores), **Semantic Search Results** (number of results returned per query), and **Reindex Duration** (time spent on reindex operations). The latency spikes around 17:00 correlate with cold starts after the Hugging Face Space wakes from sleep.

Uptime Monitoring

I added an uptime monitor for the Hugging Face Space URL with checks from multiple global regions:

AppSignal Uptime Monitoring dashboard showing the monitored URL 'https://mugdhav-mediasearchmcp.hf.space/' with check configuration including regions (Europe, Asia-Pacific, North America, South America), 2-minute warm-up period, and default email notifier. Below displays a response time graph with four colored lines representing each region, where North America shows the fastest latency at 120ms (closest to Hugging Face infrastructure), while Europe shows 822ms, helping identify region-specific versus global performance issues.

The uptime monitor configuration includes:

- **URL:** `https://mugdhav-mediasearchmcp.hf.space/`
- **Monitored from:** Europe, Asia-Pacific, North America, South America
- **Warm-up:** 2 minutes (to avoid alerting on single failed checks)
- **Notifiers:** Default email notifier

The response time graph shows latency from each region. North America at 120ms is fastest (closest to Hugging Face's infrastructure), while Europe sees 822ms. This helps identify if performance issues are region-specific or global.

Anomaly Detection Triggers

I set up a trigger for slow searches:

- **Metric:** `semantic_search.duration_ms`
- **Condition:** Goes above 5000
- **Warm-up:** 2 minutes (to avoid alerting on single slow requests)

This catches sustained performance degradation without alert fatigue from one-off spikes.

Here's what it looks like when anomalies are detected:

AppSignal Anomaly Detection Issue List displaying triggered performance alerts with a table showing multiple occurrences where transaction_duration exceeded the threshold. Peak values of 56.5 seconds and 44.37 seconds are highlighted, corresponding to index_local_directory and reindex_media operations. The right panel shows trigger configuration with condition set to greater than 200ms and notifier settings. Each occurrence row includes start/end timestamps for correlating with specific user actions or scheduled tasks.

The Anomaly Detection view shows triggered alerts with their peak values. You can see several occurrences where `transaction_duration` exceeded the threshold. Peak values of 56.5 seconds and 44.37 seconds correspond exactly to the `index_local_directory` and `reindex_media` operations I saw in the Actions view. The trigger configuration panel on the right shows the condition (> 200ms) and notifier settings. Each occurrence is tracked with start/end times, making it easy to correlate with specific user actions or scheduled tasks.

Notification Setup

In App settings → Notifications, I configured:

- **Error alerts:** "First after close" notifies on new errors, not every occurrence
- **Anomaly alerts:** Enabled for the slow search trigger

What I'm Learning from the Dashboard

Here's where it gets interesting. The AppSignal dashboard revealed performance patterns I hadn't anticipated.

The Actions View

AppSignal Performance Actions view presenting a data table with five instrumented operations and their performance metrics. Columns show Action name, Mean response time, 90th Percentile, and Throughput. Notable entries include: get_media_details at 0ms (100µs at 90th percentile), get_index_stats at 1ms, semantic_search averaging 359ms with 90th percentile at 432ms indicating a 20% variance from edge cases, and the heavy operations index_local_directory at 56.5 seconds and reindex_media at 44.37 seconds, establishing baselines for tracking optimization efforts.

The Actions view shows all my instrumented operations with mean response times and throughput:

Action	Mean	90th Percentile	Throughput
`get_media_details`	0 ms	100 µs	12
`get_index_stats`	1 ms	2 ms	2
`semantic_search`	359 ms	432 ms	16
`index_local_directory`	56.5 sec	56.5 sec	1
`reindex_media`	44.37 sec	44.37 sec	1

Key insight: `semantic_search` averages 359ms with the 90th percentile at 432ms, a 20% variance that suggests some queries hit edge cases. The metadata operations (`get_media_details`, `get_index_stats`) are lightning fast at under 2ms, which is expected since they're just dictionary lookups.

The indexing operations (`reindex_media` at 44.37 seconds, `index_local_directory` at 56.5 seconds) are the heavy hitters. Now I have baselines to track optimization efforts.

The Slow Events View

AppSignal Slow Events view displaying a performance analysis table showing specific operations consuming the most time. The table has columns for Event name, Mean time, Throughput, and Impact percentage. The critical finding highlighted: get_image_embedding operation averaging 372ms called 143 times accounts for 99.47% of slow event impact, while indexer_search at 32ms called 9 times contributes only 0.53% impact. This identifies the SigLIP image encoding as the primary bottleneck in the 56-second index_local_directory operation.

The Slow Events view surfaces the *specific operations* consuming the most time:

Event	Mean	Throughput	Impact
`get_image_embedding`	372 ms	143	99.47%
`indexer_search`	32 ms	9	0.53%

Key insight: `get_image_embedding` accounts for 99.47% of slow event impact. It's called 143 times (once per indexed image during reindex) at 372ms each. That's where the 56-second `index_local_directory` time comes from.

The SigLIP text encoder (`indexer_search` at 32ms) is much faster and only contributes 0.53% of the impact despite being called 9 times.

Host Metrics

AppSignal Host Metrics dashboard showing three time-series graphs for infrastructure monitoring: Load average spiking up to 30+ during reindexing operations indicating CPU contention with notable spikes at 13:30 and 14:30 aligning with reindex operations, CPU usage generally remaining under 5% but spiking to 60%+ during model inference, and Memory usage staying stable around 14.65 GB where the SigLIP model holds approximately 150MB with the rest being Hugging Face Space base allocation. The correlation confirms the bottleneck is CPU-bound model inference rather than memory or I/O.

The Host Metrics view shows what's happening at the infrastructure level:

- **Load average:** Spikes up to 30+ during reindexing operations, indicating CPU contention
- **CPU usage:** Generally low (under 5%) but spikes to 60%+ during model inference
- **Memory usage:** Stable around 14.65 GB. The SigLIP model holds ~150MB in memory, and the rest is the Hugging Face Space's base allocation

The correlation is clear: load average spikes (13:30, 14:30) align with reindex operations. This confirms the bottleneck is CPU-bound model inference, not memory or I/O.

Error Tracking in Action

The error tracking I implemented with `send_error()` is now catching real issues. Here's a `FileNotFoundError` that occurred when a user requested details for a file that had been moved:

AppSignal Error Summary detail page for a FileNotFoundError displaying comprehensive error context alongside the stack trace. The error message reads 'File not found: WhatsApp Image 2025-09-13 at 16.01.58_833321f0.jpg'. The interface shows parameters including source field indicating the error originated from get_media_details operation and file_path showing the missing file. A trend chart on the right displays error frequency with recent spikes, and a Saved samples section tracks multiple occurrences to distinguish between one-off issues and recurring problems.

The error detail view shows context alongside the stack trace:

- **Error message:** "File not found: WhatsApp Image 2025-09-13 at 16.01.58_833321f0.jpg"

- **Parameters:** The `source` field shows it came from `get_media_details`, and `file_path` shows which file was missing
- **Error trends:** The chart on the right shows this error spiking recently
- **Saved samples:** Multiple occurrences are tracked, making it easy to see if this is a one-off or recurring issue

The error charts view shows the pattern over time:

AppSignal Error Charts view showing a time-series line graph tracking error count over time. The graph reveals two distinct error spikes: one occurring around 16:15-16:20 and another at 17:00. The pattern indicates the errors are not random but correlate with specific user sessions or test runs. This visualization helps determine whether file access errors are steadily increasing or just occasional glitches, providing visibility that raw logs cannot offer.

Two distinct spikes are visible, one around 16:15-16:20 and another at 17:00. This pattern suggests the errors aren't random; they correlate with specific user sessions or test runs. Without this visibility, I'd be guessing at whether file access errors were increasing or just occasional glitches.

Actionable Insights

From this data, I can prioritize optimizations:

1. **Batch image embeddings:** Instead of calling `get_image_embedding` 143 times individually, I should batch them. SigLIP supports batch inference, and reducing Python loop overhead could cut reindex time significantly.

2. **Cache text embeddings:** The `indexer_search` time includes encoding the search query. For repeated queries, I could cache the text embedding.

3. **Monitor similarity score trends:** The `semantic_search.avg_similarity` gauge will show if search quality degrades as the index grows.

Bug Fixes During Instrumentation

Here are a few bugs I encountered during the instrumentation, and how I fixed them:

Initialization Order

A subtle bug: `load_dotenv()` must run *before* `Appsignal()` is initialized. Otherwise, environment variables from `.env` aren't available for local development. Small detail,

silent failure.

Configuration Auto-Discovery

My initial setup used `__appsignal__.py` with the standard `appsignal.start()` pattern. AppSignal started but wasn't sending data. The issue: the `active` option defaults to `false`, and if auto-discovery silently fails, nothing gets reported.

The fix was inline configuration with explicit `os.getenv()` reads, removing the auto-discovery dependency.

BaseModelOutputWithPooling

Newer versions of the `transformers` library return a `BaseModelOutputWithPooling` object from `get_image_features()` instead of a raw tensor. This broke embedding normalization:

```
# Before (broke on newer transformers)
image_features = self.model.get_image_features(**inputs)
image_features = image_features / image_features.norm(dim=-1, keepdim=True)

# After (works with both old and new)
image_features = self.model.get_image_features(**inputs)
if not isinstance(image_features, torch.Tensor):
    image_features = image_features.pooler_output
image_features = image_features / image_features.norm(dim=-1, keepdim=True)
```

AppSignal would have caught this error in production with a full stack trace, which is exactly why I added error tracking before deploying.

What's Next

With the baseline established, I'm planning to:

1. **Add a custom dashboard** for search quality metrics (similarity scores over time)
2. **Set up anomaly detection** on `index.total_files` to catch if the index silently loses data
3. **Track model load time** as a separate gauge to monitor cold starts on the Hugging Face Space

The server is live at mugdhav-mediasearchmcp.hf.space, and now I can actually see what's happening under the hood.